

EquityRoots NSE

System Architecture & Scoring Design

Low-Level Design Reference · Internal Use Only

ETL · Python

Postgres (local)

Neon Cloud

Next.js · Vercel

Scoring Engine

2

Full Data Pipeline

End-to-end: Screener → ETL → Postgres → Neon → Vercel

3

Scoring Engine — Part 1

Metric computation · peer bucket assignment
· percentile ranking

4

Scoring Engine — Part 2

Pillar scores · Industry Score (re-percentiled) ·
Your Score (custom blend)

5

Scorecard Structure

42 cluster scorecards · 4 maturity tiers · loss-
maker fallbacks

6

Formula Library

37 active formulas across Quality, Valuation,
Momentum pillars

7

Web Layer — Routes

9 app routes + key components (Controls,
MetricViz, score.ts ...)

8

Database Schema

app.scores · metrics_snapshot ·
cluster_scorecard_active · screener_meta

9

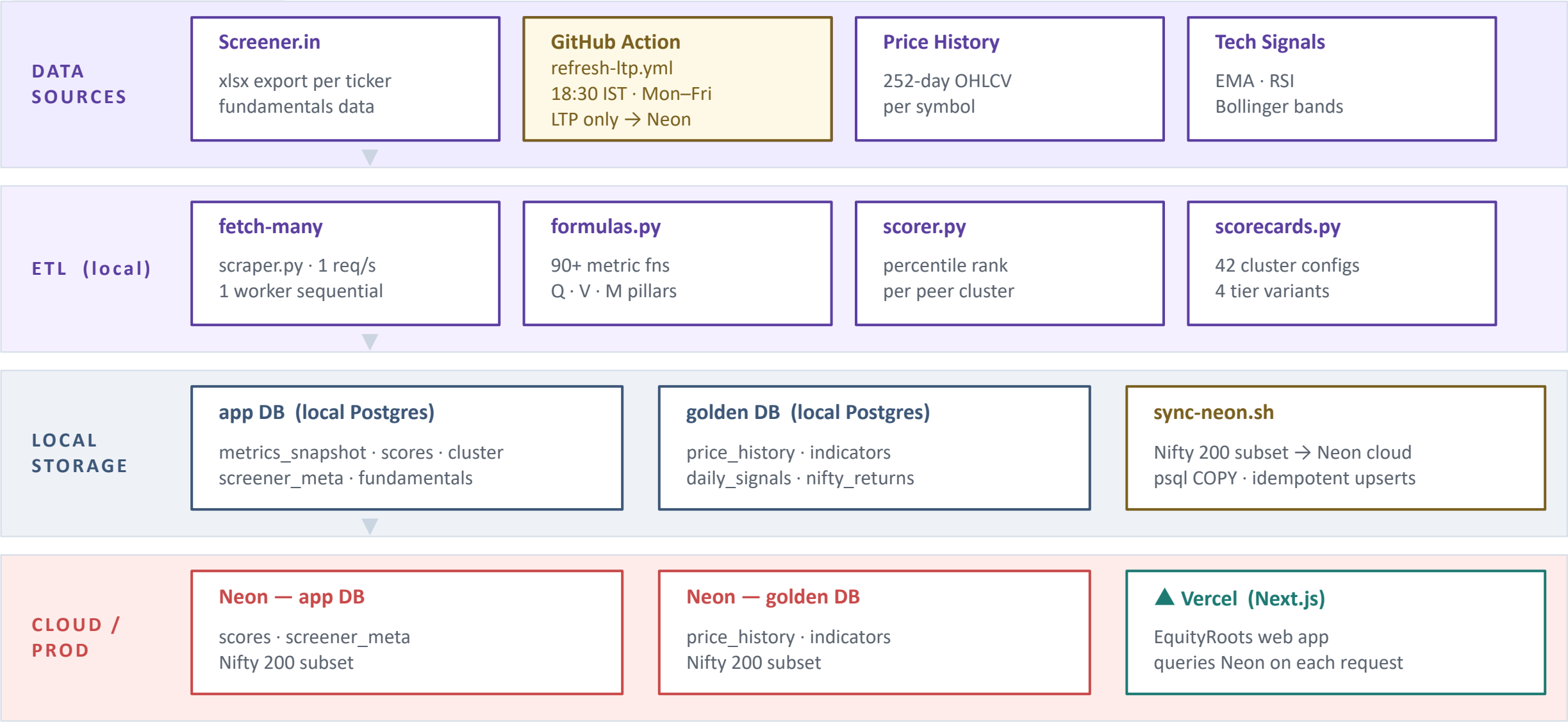
Score Bands & Comparison

5 display bands · Industry Score vs Your Score
side-by-side

10

Data Flow Reference

Every hop in the pipeline with frequency and
transport method



1

Metric Computation

metrics.py loads annual and quarterly financials from app.fundamentals_* and 252-day price history from golden.price_history. It calls every formula in the cluster's active scorecard and writes raw values to app.metrics_snapshot as a JSONB dict. Nifty returns (3m/6m/12m) are computed as market benchmark for relative momentum formulas.

2

Peer Bucket Assignment

Each stock is placed in a peer bucket by (cluster_id, maturity_tier). Scores are always peer-relative — not market-wide. If the primary bucket has fewer than 10 peers the system falls back to broader groups. score_status records which bucket was used.

app.metrics_snapshot · cluster_metrics JSONB

Slide 3 of 10

```
{
  "roe_3y":      18.4,
  "roe_5y":      16.1,
  "pe_ttm":      null,    ← loss-maker
  "ev_sales_ttm": 3.2,    ← fallback
  "ret_12m_rel": 12.3,
  "op_margin_3y": 22.1,
  ...90+ fields per stock
}
```

full

(cluster, tier) bucket ≥10 peers — best accuracy

partial-cluster-mixed-tiers

All tiers merged into one cluster bucket — ≥10 found

partial-meta-cluster

Fell back to meta-cluster + tier bucket — thinnest cluster

Slide 3 of 10 (cont.)

3

Percentile Ranking

For each formula, raw values within the peer bucket are converted to a 0–100 percentile rank. Best stock = 100, worst = 0. Null values stay null — a stock is never penalized for missing data. The direction decorator determines which end is 'better':

↑ **@_higher** → larger value = better rank (100)

Examples:

```
roe_3y · roce_3y · op_margin_3y · rev_cagr_5y
ret_12m_rel · np_cagr_5y · fcf_yield
```

↓ **@_lower** → smaller value = better rank (100)

Examples:

```
pe_ttm · pb · ev_ebitda_ttm · debt_equity
net_debt_ebitda · wc_days · capex_intensity_3y
```

scorer.py · `_percentile_rank()`

```
def _percentile_rank(values: list[float | None], higher_is_better: bool) -> list[float | None]:
    valid = [(i, v) for i, v in enumerate(values) if v is not None]
    n = len(valid)
    sorted_valid = sorted(valid, key=lambda x: x[1], reverse=higher_is_better)
    # higher_is_better=True → descending sort → first item gets rank 100 (best)
    # higher_is_better=False → ascending sort → first item gets rank 100 (cheapest = best)
    ranks = {}
    for rank, (i, _) in enumerate(sorted_valid):
        ranks[i] = round((1 - rank / (n - 1)) * 100) if n > 1 else 50
    return [ranks.get(i) for i in range(len(values))] # None stays None
```

4 Weighted Pillar Scores

Component percentiles are weighted within each pillar (Quality, Valuation, Momentum). Weights re-normalize across non-null components — missing data does NOT collapse the pillar score.

$$\text{quality_pct} = \frac{\sum(\text{component_pct}[k] \times w[k])}{\sum(w[k] \text{ for all non-null } k)}$$
5 Industry Score (composite_pct)

Cluster-default pillar weights blend the three pillars into a raw composite. That raw composite is then RE-PERCENTILED within the peer bucket → composite_pct. Double-percentile = uniform 0-100 regardless of pillar skew. Stored in app.scores.

```
raw = Q_pct*w_q + V_pct*w_v +
M_pct*w_m
composite_pct =
percentile_rank(raw)
                    within (cluster,
                    tier)
```

6 Your Score (blend — on /discover)

Stored pillar percentiles are re-blended in SQL using slider weights. No re-percentiling — instant from stored values. Sliders auto-renormalize so w_q + w_v + w_m = 100 always.

```
blend = ROUND(
  ( quality_pct  × :w_q
    + valuation_pct × :w_v
    + momentum_pct × :w_m ) / 100
) -- SQL, per request
```

scorecards.py — Scorecard dataclass + example

```

@dataclass
class Scorecard:
    pillar_weights: dict[str, float] #
    {"q":50,"v":30,"m":20}
    quality: dict[str, float] # formula_id →
    intra-pillar weight
    valuation: dict[str, float] # each dict sums
    to 100
    momentum: dict[str, float]
    loss_maker_val_fallback: list[tuple] #
    [("pb",1.0)]

# — Example: Private Banks —————
"bfsi_pvt_banks": Scorecard(
    pillar_weights = {"q":50, "v":30, "m":20},
    quality = {
        "roa_3y":25, "roe_3y":18,
        "book_value_cagr_5y":18,
        "loan_book_cagr_3y":14,
        "np_cagr_5y":12,
        "interest_coverage":8,
        "net_npa":5,
    },
    valuation = {
        "pb":50, "pe_ttm":25,
        "ev_ebitda_ttm":15, "div_yield":10,
    },
    loss_maker_val_fallback = [("pb", 1.0)],
)

```

Slide 5 of 10

Maturity Tiers

VETERAN	10+ years listed +5% Quality (from Momentum) 10y CAGRs · roe_avg_above_threshold_10y
MATURE	Default — most NSE stocks Base scorecard as defined in scorecards.py No adjustments
MID	Mid-sized companies −5% Quality, +5% Momentum 3y CAGRs replace 5y versions
NEW	Recently listed / thin data −15% Quality, +15% Momentum Drops CAGRs · uses latest values only

QUALITY

22 formulas

Profitability

$$\text{roe_3y} \cdot \text{roe_5y} \cdot \text{roa_3y} \cdot \text{roce_3y} \cdot \text{roce_5y}$$

Margins

$$\text{op_margin_3y} \cdot \text{op_margin_latest} \cdot \text{op_margin_trend} \cdot \text{ebitda_margin_3y}$$

Growth (CAGR)

$$\text{rev_cagr_3y/5y/7y/10y} \cdot \text{np_cagr_3y/5y/7y/10y}$$

Consistency

$$\text{np_consistency_3y/5y} \cdot \text{roe_avg_above_threshold_5y/10y}$$

Cash Flow Quality

$$\text{cfo_pat_3y} \cdot \text{cfo_ebitda_3y} \cdot \text{cfo_sales_3y}$$

Leverage

$$\text{debt_equity} \cdot \text{interest_coverage} \cdot \text{equity_to_assets}$$

VALUATION

7 formulas

Core

$$\text{pe_ttm} \cdot \text{pb} \cdot \text{ev_ebitda_ttm}$$

Growth-adjusted

$$\text{peg} \cdot \text{earnings_yield_trend}$$

Cash return

$$\text{fcf_yield} \cdot \text{div_yield}$$

Loss-maker fallbacks

$$\text{ev_sales_ttm} \\ \text{p_aum} \quad (\text{NBFCs}) \\ \text{p_premium} \quad (\text{insurance})$$

MOMENTUM

8 formulas

Market-relative

$$\text{ret_3m_rel} \\ \text{ret_6m_rel} \\ \text{ret_12m_rel}$$

Technical

$$\text{pct_above_200ema_252d} \\ \text{ema_stack_bull} \\ \text{tech_net_score_scaled}$$

Earnings momentum

$$\text{sales_yoy_q} \cdot \text{np_yoy_q}$$

App Routes

/discover Screener table · Industry Score + Your Score · weight sliders
/sectors Cluster heatmap · 1W/1M/1Y returns · peer-average scores
/stock/[symbol] Stock detail · 4 tabs · spider chart · latest result card
/industry/[id] Peer table for one cluster · composite ranking
/glossary 52 metric entries · animated MetricViz gauge examples
/compare Side-by-side 2–5 stocks on same scorecard
/about Methodology walkthrough · animated pipeline diagrams
/admin/scorecards Edit pillar weights & formula weights per cluster

Key Components

Slide 7 of 10

Controls.tsx Q/V/M weight sliders · auto-renormalization · 4 presets · min-score filters
SnapshotRibbon.tsx Dark sticky 28px bar · snapshot date · coverage · movers · 1h cache
StockPageTabs.tsx Client tab switcher · Latest result / About / Strengths / Numbers tabs
MetricViz.tsx use client · IntersectionObserver · count-up animation · 5-tier gauge
score.ts band() · bandColor() · fmtPct() · tierLabel() — shared across all routes
spider.ts 6D radar chart data builder — Quality/Valuation/Momentum + sub-pillars
explainer.ts Rule-based narrative sentences for each pillar score band
db.ts Postgres connection pools — app pool + golden pool (Neon over TLS)

app.scores

app DB — Neon cloud

symbol, snapshot_date	TEXT, DATE
cluster_id	TEXT — which cluster peer group
maturity_tier	TEXT — veteran / mature / mid / new
quality_pct	NUMERIC 0-100 — within-cluster percentile
valuation_pct	NUMERIC 0-100
momentum_pct	NUMERIC 0-100
composite_pct	NUMERIC 0-100 — Industry Score (re-percentiled blend)
quality_components	JSONB — {formula_id: pct_value}
valuation_components	JSONB

app.cluster_scorecard_active

app DB — admin-editable via /admin/scorecards

cluster_id	TEXT PRIMARY KEY
pillar_weights	JSONB — {"q":50,"v":30,"m":20}
quality	JSONB — {formula_id: weight}
valuation	JSONB
momentum	JSONB
loss_maker_val_fallback	JSONB — e.g. [(pb, 1.0)]

app.metrics_snapshot

app DB — local + Neon

symbol, snapshot_date	TEXT, DATE
maturity_tier	TEXT
cluster_metrics	JSONB — {formula_id: raw_value_or_null}
market_cap	NUMERIC
current_price	NUMERIC
score_status	TEXT

app.screener_meta

app DB — updated daily by GitHub Action

symbol	TEXT PRIMARY KEY
current_price	NUMERIC — LTP from NSE bhavcopy
market_cap_cr	NUMERIC
face_value	NUMERIC
no_of_shares	BIGINT

Score Bands (0 – 100)

Excellent ≥ 80	Top 20% of peers in cluster
Good 60–79	Above average in peer group
Neutral 40–59	Middle of the pack
Weak 20–39	Below average peer ranking
Poor < 20	Bottom 20% of peers

Industry Score vs Your Score

	Industry Score	Your Score
Weights	Cluster-tuned e.g. Banks: Q50/V30/ M20	Your slider setting e.g. Q30/V50/M20
Re-percentiled	Yes — re-ranked within peer bucket (uniform 0-100)	No — direct weighted average of pillar pct
Stored in DB	Yes — composite_pct in app.scores	No — SQL per-request from stored pillars
Computed by	ETL scorer.py (batch, weekly)	SQL at query time (instant)
Use case	Platform consensus: rank in cluster	Your lens: excel on what you value?

From	To	What moves	How	Frequency
Screener.in	local app DB	Annual + quarterly financials	Python httpx · xlsx parse	Weekly manual
NSE Bhavcopy	Neon app DB	Current LTP per symbol	GitHub Action refresh-ltp.yml	Daily 18:30 IST
local app DB	local golden DB	Price signals · 200 EMA	Pre-computed by signals script	Weekly manual
local app DB	Neon (app)	Scores + meta — Nifty 200	sync-neon.sh (psql COPY)	After ETL run
local golden DB	Neon (golden)	Price history — Nifty 200	sync-neon.sh (psql COPY)	After ETL run
Neon cloud	Next.js server	Scores · prices · fundamentals	postgres.js over TLS	Per page request
Next.js server	Browser	HTML + hydration data	Vercel edge CDN · revalidate cache	Cached per route

Note: Full ETL (./fetch) is weekly manual (~60 min for 2,150 stocks). Current prices (LTP) refresh automatically daily via GitHub Action. Historical scores do not update between ETL runs.

Quick Reference

CLI commands · reference documents · key files

Commands

<code>./fetch</code>	# Full universe ETL (~60 min, 2150 stocks, 1 req/s)
<code>./fetch --limit 100</code>	# Smoke test – first 100 stocks from queue
<code>./fetch --only TCS,INFY,HDFC</code>	# Specific symbols only
<code>./snap</code>	# Score latest metrics snapshot (~2 min)
<code>scripts/sync-neon.sh</code>	# Sync Nifty 200 subset to Neon production

Reference Documents

→ docs/SCORING_DESIGN.md	– full logic · formulas · DB tables · trade-offs
→ docs/SCORECARDS_BY_INDUSTRY.md	– all 42 cluster scorecards
→ etl/src/fundamental_etl/scoring/	– scorer.py · formulas.py · metrics.py · scorecards.py
→ web/src/app/discover/	– Controls.tsx · types.ts · page.tsx